Stewer Gabriel

DOCUMENTAÇÃO DO SISTEMA

Plano e relatório de *testes*

Caixa branca, cinza e preta, segurança, desempenho, rastreabilidade e achados.

Site e painel de stewergabriel.com
Visagismo e consultoria de imagem · Palmas, Tocantins · Desenvolvido por Gustavo Dev

Versão 2.1.0
08 de novembro de 2026

STEWER GABRIEL

Plano e relatório de testes

Caixa branca, cinza e preta, segurança, desempenho, rastreabilidade e achados.

Como o sistema é verificado, o que cada suíte cobre e como cada requisito de requisitos.md é testado.

Resumo

SUÍTE	TÉCNICA	O QUE RODA	TESTES
Unitários	caixa branca	bibliotecas da API e do site, isoladas	233
Banco	caixa branca	migrations aplicadas do zero num SQLite em memória	53
API	caixa preta e caixa cinza	a API real (wrangler, runtime da Cloudflare) com um D1 isolado	210
Segurança	caixa preta e caixa cinza	OWASP Top 10 contra a API real: SQL injection, sessão, cabeçalhos, SSRF, upload	318
Desempenho	caixa branca	orçamento de peso do build (`dist/`)	16
E2E	caixa preta	site e painel num navegador, no computador (1440px) e no celular (390px)	118
Total automatizado			**948**
Carga e estresse	caixa preta	autocannon contra a API real, até 100 conexões	7 cenários
Lighthouse	caixa preta	celular e computador, três rodadas cada, na CI	2 telas

Os E2E rodam duas vezes cada, uma no computador e uma no celular: são 59 cenários (dois deles valem só para uma das telas e pulam a outra). Entre os E2E estão os de acessibilidade (axe-core, WCAG 2 A e AA), os de segurança no navegador, os Core Web Vitals, a abertura de marca e o movimento de rolagem.

Cobertura de linhas dos unitários: 88% (98% em `src/lib`, 80% em `functions/_lib`). O que fica de fora do unitário depende do banco (sessões, `conteudo.ts`, `auditoria.ts`) e é coberto pelas suítes de API e segurança.

As três técnicas

Caixa branca conhece o código e testa os caminhos de dentro: cada ramo da validação, o hash de senha, a assinatura do token, a leitura das coordenadas, a troca do conteúdo de reserva, o formato binário do zip. Fica em `tests/unit/` e `tests/db/`.

Caixa preta só conhece a interface: manda um pedido HTTP ou clica na tela e confere a resposta, sem saber como foi feito. É a maior parte de `tests/api/` e todo o `tests/e2e/`. Os contratos em `tests/contratos.ts` (schemas Zod) definem o que a API promete ao site e ao painel.

Caixa cinza faz o pedido pela interface, como a preta, mas depois abre o banco do servidor de teste (só leitura) e confere o que foi gravado: que a senha virou hash, que a auditoria registrou quem fez, que a versão subiu, que o dado inválido não chegou ao banco, que nenhuma tabela mudou depois de 215 tentativas de SQL injection, que "Sair" apagou a sessão. Os testes marcados "caixa cinza" em `tests/api/` e `tests/seguranca/` fazem isso pela função `banco()` de `tests/api/cliente.ts`.

Além das três técnicas, duas frentes têm suíte própria:

Segurança ataca a API e o navegador como alguém mal-intencionado que conhece o sistema (inclusive com o `JWT_SECRET` vazado), organizada pelo OWASP Top 10. O resumo por risco está em `SECURITY.md`.

Desempenho mede em três camadas: o peso do que o build gera (orçamento), o que o navegador sente com a rede e a CPU limitadas (Core Web Vitals, como o Lighthouse) e o quanto a API aguenta (carga e estresse).

Ambiente

Nenhum teste toca o banco de desenvolvimento nem o de produção. `tests/servidor-teste.mjs` cria, a cada rodada, uma pasta temporária, aplica todas as migrations, cria um usuário de teste e sobe o `wrangler pages dev` apontado para ela. Os testes de API e os E2E usam esse servidor; ao terminar, a pasta é apagada.

Os arquivos das suítes de API e segurança contam uma história em ordem (01-contrato até 13-entradas) sobre um banco só, por isso rodam em sequência, pelo nome. Os E2E que mudam conteúdo devolvem tudo como estava, porque o computador e o celular rodam sobre o mesmo banco.

Como rodar

```
npm run build            # a API e os E2E servem o dist/
npm test                 # unitários + banco + API + segurança + orçamento de peso
npm run test:e2e         # E2E (baixe o navegador uma vez: npx playwright install chromium)
npm run test:seguranca   # só a segurança: API (OWASP) e navegador
npm run test:desempenho  # só o desempenho: orçamento e Core Web Vitals
npm run test:carga       # carga e estresse da API (gera tests/relatorio-carga/)
npm run auditoria        # npm audit das dependências
npm run test:tudo        # tudo, na ordem
npm run test:cobertura
```

No GitHub, três workflows rodam a cada push e pull request na `main`:

- `testes.yml`: auditoria de dependências, tipos, orçamento de peso, unitários, banco, API, segurança, carga e estresse e E2E; guarda o relatório do E2E quando algo falha e publica a tabela de carga no resumo do job;
- `lighthouse.yml`: Lighthouse no celular e no computador, com os tetos de `tests/lighthouse`;
- `codeql.yml`: análise estática de segurança do GitHub (também toda segunda-feira).

O Dependabot abre pull request quando sai atualização de dependência.

O que cada suíte cobre

Unitários (`tests/unit/`)

ARQUIVO	TESTES	O QUE CONFERE
`auth.test.ts`	34	hash PBKDF2 com salt, compatível com o script de criar acesso; JWT assinado, adulterado, vencido, `alg: none`, outro algoritmo, carga fora do formato; cookie seguro
`http.test.ts`	26	leitura do corpo com limite e tipo; formato único de erro; erro inesperado sem detalhe; id de rota só com dígitos (`0x1`, `1e3`, `Infinity` recusados)
`entidades.test.ts`	32	lista fechada de tabelas (inclusive `__proto__` e `toString`); limites; enquadramento da foto; o que mudou, em português
`config.test.ts`	38	WhatsApp, Instagram, mensagem e coordenadas no formato que o Stewer digita ou cola, inclusive o link longo do Google Maps; link curto pede para ser aberto; ponto fora do Brasil
`coordenadas.test.ts`	27	o ponto em cada formato de link do Google Maps (alfinete, `?q=`, `/search/`, `@`); prioridade entre eles; vírgula decimal; arredondamento
`link-mapa.test.ts`	16	o resolvedor do link curto: só hosts do Google, só HTTPS, sem usuário nem porta, no máximo 6 saltos, página de consentimento; nada vai para a rede (fetch falso)
`mapa.test.ts`	9	a ordem das fontes do mapa (CARTO, OpenFreeMap, OSM), quando desistir de uma, o crédito de cada uma
`movimento.test.ts`	12	o número que a contagem usa ("+800", "10+", "1.500"): o último quadro é igual ao texto original
`site.test.ts`	7	telefone formatado, links de WhatsApp, Instagram e rota; graus, minutos e segundos iguais aos do site original
`textos.test.ts`	10	chaves únicas; todas as seções; texto original cabe no limite; aplicar ignora chave e tipo errados; só o endereço é editado na página de contatos
`conteudo-remoto.test.ts`	12	troca antes do render, mesma referência das listas, vídeos intactos, foto quebrada fora, abertura sem foto, API fora ou lenta
`utilitarios.test.ts`	10	zip do backup lido byte a byte e conferido com o CRC do zlib; nomes de arquivo; senha temporária

Banco (`tests/db/`)

`schema.test.ts` (53): migrations em sequência e aplicáveis uma a uma; todas as tabelas; o seed igual ao conteúdo do código, item a item e na mesma ordem; toda foto do seed existe em `public/`; CHECK, NOT NULL, UNIQUE e FOREIGN KEY de cada lista; sessão de pessoa inexistente recusada e sessões apagadas junto com a pessoa; versão para edição simultânea; índices e o plano da consulta da auditoria.

API (`tests/api/`)

ARQUIVO	TESTES	O QUE CONFERE
`01-contrato`	10	`/api/conteudo` no contrato, sem nada interno, `no-cache`; formato único de erro; foto original redireciona
`02-autenticacao`	64	login, trava, hash no banco, sessão, logout, e as 51 rotas do painel exigindo login
`03-listas`	42	as sete listas no contrato; criar, editar, ocultar, reordenar, apagar refletindo no site; ordem incompleta; validação; lista fechada
`04-fotos`	18	tipos aceitos pelos bytes; arquivo disfarçado, SVG e grande demais recusados; download; foto em uso protegida (lista e texto)
`05-contatos`	22	WhatsApp, Instagram, mensagem e mapa chegando ao site; link do Google Maps (longo e curto); inválidos; pacote inteiro ou nada
`06-textos`	13	gravar, foto de seção, voltar ao original, normalização, limites, pacote inteiro ou nada
`07-concorrencia`	6	versão velha recusada com o nome de quem salvou; dez gravações ao mesmo tempo, uma vence
`08-usuarios-auditoria`	18	dar acesso, senha temporária (encerra a sessão aberta), troca obrigatória, tirar acesso, auditoria, espaço do banco
`09-seguranca`	17	CSRF, XSS guardado como texto, Instagram malicioso, corpo grande, tipo errado, prototype pollution, erro sem vazar detalhe

Segurança (`tests/seguranca/` , OWASP Top 10)

ARQUIVO	TESTES	O QUE CONFERE
`10-sql-injection`	215	15 payloads (tautologia, `UNION`, comando empilhado, comentário, blind por tempo, `PRAGMA`, `sqlite_master`) em login, rotas, ids, textos, contatos, auditoria, usuários e link do mapa; nunca 500, nunca dado do banco na resposta; caixa cinza: nenhuma tabela mudou e o banco continua íntegro
`11-sessao`	23	token forjado: `alg: none`, carga trocada, outro segredo, vencido, `JWT_SECRET` vazado sem sessão aberta ou com a sessão de outra pessoa; cookie malformado; "Sair" mata o cookie copiado e não derruba o outro aparelho; troca de senha e senha temporária encerram as outras sessões; acesso removido vale na hora
`12-cabecalhos`	29	CSP diretiva a diretiva (nada de `*`, `unsafe-eval` ou origem a mais), `nosniff`, `X-Frame-Options`, HSTS, COOP, Permissions-Policy nas páginas e na API, inclusive em erro e em foto; o build sem script inline; painel fora do Google; rota de API inexistente em JSON; CORS fechado
`13-entradas`	51	SSRF no link do mapa (metadados da nuvem, localhost, rede interna, `file:`, `user@`, porta, domínio parecido); upload de HTML, SVG, executável e PDF disfarçados; poliglota servido como imagem que não executa; header injection no nome do download; path traversal; mass assignment

No navegador, `seguranca.spec.ts` confere que o navegador obedece: script injetado e script de outro site não rodam, o painel não abre em iframe de outro site, o cookie é invisível ao JavaScript e não vale depois de "Sair", e XSS guardado aparece como texto. E todo E2E tem um **fiscal da CSP**: qualquer coisa que a política bloquear durante qualquer teste reprova o teste, para a CSP nunca quebrar o site em silêncio.

Desempenho

Orçamento de peso (`tests/desempenho/orcamento.test.ts`, 16): confere o `dist/`, o que o celular baixa de verdade, em gzip.

O QUÊ	TETO	HOJE
JavaScript da abertura do site	65 KB	~58 KB
CSS da abertura do site	9 KB	~6 KB
HTML	2 KB	~1 KB
JavaScript da abertura do painel	72 KB	~63 KB
foto de `public/`	250 KB cada	maior: 217 KB

Também: o mapa e o painel fora da abertura do site; nada de outro servidor bloqueando a primeira pintura; o MapLibre como único arquivo grande (e sob demanda); só fontes do subconjunto latino.

Core Web Vitals (`tests/e2e/desempenho.spec.ts`): no navegador, com as condições do Lighthouse (celular: 4G lento, 150 ms e 1,6 Mbps, CPU 4× mais lenta; computador: cabo), como um visitante novo, com a abertura de marca e o movimento ligados.

MÉTRICA	TETO (O "BOM" DO GOOGLE)	CELULAR	COMPUTADOR
FCP	1,8 s	~0,7 s	~0,25 s
LCP	2,5 s	~1,7–2,2 s	~0,5 s
TBT	200 ms	~70 ms	0 ms
CLS (rolando a página toda)	0,1	0	0
INP (abrir uma foto)	200 ms	~40 ms	~15 ms
p95 de `/api/conteudo`	300 ms	~15 ms	~15 ms
painel do login à tela pronta, no 4G lento	3 s	~2 s	~2 s

Também: o Leaflet só baixa perto do mapa e nenhum pedido sai para outro servidor na abertura.

Carga e estresse (`tests/carga/carga.mjs`, autocannon), contra o servidor de teste (um processo só, na máquina da CI: os números da Cloudflare são melhores; isto pega regressão, não promete latência).

CENÁRIO	CONEXÕES	PEDIDOS/S	P50	P97,5	5XX
conteúdo do site	10	~380	24 ms	41 ms	0
página inicial	10	~370	25 ms	45 ms	0
painel (com login)	5	~270	16 ms	40 ms	0
estresse	25	~440	51 ms	97 ms	0
estresse	50	~440	102 ms	233 ms	0
estresse	100	~425	221 ms	326 ms	0
força bruta: 200 senhas erradas na mesma conta	10	—	36 ms	243 ms	0

Lighthouse (CI, três rodadas por tela): hoje 95 / 100 / 100 / 100 no celular e 98 / 100 / 100 / 100 no computador (desempenho, acessibilidade, boas práticas, SEO). Os tetos estão em `tests/lighthouse/LEIAME.md`.

E2E (`tests/e2e/`)

ARQUIVO	CENÁRIOS	O QUE CONFERE
`site.spec.ts`	11	site abre sem erro com o banco; portfólio intercala vídeo; foto ampliada pelo teclado; API fora e lenta; links de contato; mapa (e as reservas dele); seção some sem itens; HTML no nome do lugar não vira elemento
`painel.spec.ts`	17	login; editar serviço e ver no site; reordenar com as setas; texto e voltar ao original; enviar foto (reduzida para 1600px); nova foto no portfólio; contatos; colar link do Google Maps e arrastar o alfinete; endereço; erro em português; duas pessoas no mesmo item; acesso com senha temporária; auditoria; espaço; todas as abas sem rolagem lateral; sair
`acessibilidade.spec.ts`	4	login, todas as abas e o formulário aberto sem violação grave; o que o painel coloca no site sem violação nova
`seguranca.spec.ts`	6	ver "Segurança" acima
`desempenho.spec.ts`	7	ver "Desempenho" acima
`abertura.spec.ts`	6	a abertura de marca aparece para visitante novo, trava a rolagem por trás, sobe sozinha, espera a API lenta, não prende o site quebrado, não repete na sessão, não aparece para quem pede menos movimento
`movimento.spec.ts`	8	o GSAP só baixa quando a pessoa rola; título dividido em linhas sobe e mantém o nome acessível; foto se abre e devolve o hover; número conta até o valor e o leitor de tela lê o final; o que já estava na tela não some; parallax só no computador; movimento reduzido não baixa nada

Os E2E rodam com `prefers-reduced-motion: reduce`, para olhar o resultado e não a coreografia; `abertura`, `movimento` e `desempenho` ligam o movimento.

Rastreabilidade

REQUISITO	TESTADO EM
RF01	`conteudo-remoto` · `01-contrato` · `03-listas` · `site.spec`
RF02	`conteudo-remoto` · `03-listas` (ocultar, tudo oculto) · `site.spec` (seção some)
RF03	`conteudo-remoto` (falhas e 2,5 s) · `site.spec` (API fora e lenta)
RF04	`textos` · `06-textos` · `painel.spec`
RF05	`site` · `conteudo-remoto` · `05-contatos` · `site.spec` (links e mapa)
RF06	`conteudo-remoto` (vídeos intactos) · `site.spec` (vídeo intercalado)
RF07	`mapa` · `site.spec` (OpenFreeMap sem chave, OSM com o estilo fora) · `desempenho.spec` (mapa sob demanda)
RF08	`abertura.spec` · `movimento.spec` · `movimento` · `desempenho.spec` (CLS com movimento)
RF09	`site.spec` (mapa pela âncora) · `movimento.spec` (âncora antes do movimento)
RF10	`02-autenticacao` · `painel.spec`
RF11	`entidades` · `03-listas` · `painel.spec`
RF12	`utilitarios` (zip) · `04-fotos` · `painel.spec`
RF13	`painel.spec` (3000px vira 1600px)
RF14	`textos` · `06-textos` · `painel.spec`
RF15	`config` · `coordenadas` · `link-mapa` · `05-contatos` · `08-usuarios-auditoria` (senha) · `painel.spec`
RF16	`utilitarios` (senha temporária) · `08-usuarios-auditoria` · `painel.spec`
RF17	`08-usuarios-auditoria` · `painel.spec`
RF18	`entidades` (o que mudou) · todas as suítes de API (`ultimaAuditoria`) · `painel.spec`
RF19	`08-usuarios-auditoria` · `painel.spec`
RF20	`03-listas` · `painel.spec`
RNF01	`auth` · `02-autenticacao` (caixa cinza) · `08-usuarios-auditoria`
RNF02	`auth` · `schema` (sessões) · `02-autenticacao` · `11-sessao` · `seguranca.spec`
RNF03	`09-seguranca` · `12-cabecalhos` · `seguranca.spec` · fiscal da CSP em todos os E2E
RNF04	`02-autenticacao`
RNF05	`entidades` · `03-listas` · `09-seguranca` · `10-sql-injection`

REQUISITO	TESTADO EM
RNF06	`04-fotos` · `13-entradas`
RNF07	`schema` · `07-concorrencia` · `painel.spec`
RNF08	`05-contatos` · `06-textos` · `03-listas` (ordem)
RNF09	`entidades` · `config` · `schema` · `03-listas` · `04-fotos`
RNF10	`contratos.ts` em todas as suítes de API · `http`
RNF11	`painel.spec` (todas as abas no celular e no computador)
RNF12	`acessibilidade.spec` · `movimento.spec` (nome acessível) · Lighthouse
RNF13	`schema` (índices) · `08-usuarios-auditoria` (limpeza e espaço)
RNF14	`conteudo-remoto` · `site.spec`
RNF15	`schema` · workflow do CI
RNF16	`schema` (seed igual ao código) · `site` (coordenadas iguais) · comparação de capturas (abaixo)
RNF17	`link-mapa` · `13-entradas` · `12-cabecalhos` · `npm audit` e CodeQL na CI
RNF18	`orcamento` · `desempenho.spec` · `movimento.spec` (GSAP fora da abertura) · Lighthouse
RNF19	`desempenho.spec` (p95 e painel) · `carga.mjs`

Achados

O que os testes encontraram durante o desenvolvimento e já foi corrigido:

ACHADO	COMO APARECEU	CORREÇÃO
Instagram `a/../../x` era aceito e gravado como `a`, sem avisar	`09-seguranca`	só limpa o resto do texto quando é o link do perfil; qualquer outra barra é recusada
Títulos dos grupos da lateral do painel com contraste abaixo do AA	`acessibilidade.spec`	cor trocada pelo tom "areia" do painel
Os números de "Quem é" quebravam diferente no celular depois da troca para lista	comparação de capturas	volta à mesma estrutura de flex do original
Os botões de zoom do mapa perdiam o estilo escuro	comparação de capturas	o CSS do Leaflet voltou a carregar antes do CSS do site
Ordem enviada sem um dos itens deixaria dois itens na mesma posição	revisão ao escrever `03-listas`	a API exige a lista completa e responde 409

ACHADO	COMO APARECEU	CORREÇÃO
"Sair" só apagava o cookie: um token copiado antes continuava entrando por 8 horas	`11-sessao`	sessões gravadas no banco (migração 0003): sair, trocar a senha e senha temporária encerram
`?usuario=abc` na auditoria virava `NaN` e o filtro sumia, devolvendo a auditoria de todo mundo	`10-sql-injection`	o filtro é validado e responde 400
O id da rota aceitava `0x1` e `1e3` como número	`13-entradas`	só dígitos
Rota de API inexistente respondia 200 com o HTML do site	`12-cabecalhos`	`/api/[[rota]]` responde 404 (ou 405) em JSON
A cortina da abertura nunca subia se o JavaScript do site falhasse ao carregar	`abertura.spec`	falha ao montar também libera a cortina
Títulos em linhas eram lidos colados por leitor de tela e pelo Google ("StewerGabriel")	`movimento.spec` (nome acessível)	espaço antes de cada quebra de linha
Link com âncora (`#onde-atende`) abria no topo, porque a seção ainda não existia	`site.spec` , depois que o mapa passou a carregar sob demanda	a página vai até a âncora depois de montar
A área do mapa tinha `aria-label` sem papel antes de o Leaflet chegar	`acessibilidade.spec`	`role="region"`
"Scroll" da abertura com contraste 1,7:1	Lighthouse	cor areia (5,2:1)
`robots.txt` inexistente: o endereço devolvia o HTML e o Google lia 36 erros	Lighthouse	`robots.txt` e `sitemap.xml`
LCP de 3,2 s no celular: as três fotos da abertura e as fontes dividiam a banda, e a foto só era pedida depois de o React montar	`desempenho.spec`	primeira foto com prioridade e as outras depois; conteúdo pré-carregado com o HTML; entrada sem React (LCP ~1,7 s)
Fontes vinham de um CSS bloqueante do Google Fonts	`orcamento`	fontes servidas pelo próprio site
Leaflet (50 KB) na abertura para um mapa no fim da página	`orcamento`	mapa sob demanda
Foto do portfólio com 871 KB	`orcamento`	JPEG de 96 KB (migração 0004)
O GSAP travava o processador do celular por ~250 ms logo depois de carregar	`desempenho.spec` (TBT)	só baixa quando a pessoa rola, e se prepara em pedaços
Montar a página inteira de uma vez travava ~200 ms, congelando o S da abertura	`desempenho.spec` (TBT)	o resto da página monta numa transição do React

Fidelidade ao design (RNF16)

Além dos testes automáticos, o site novo foi comparado com o site da `main` por captura da página inteira, no computador (1440px) e no celular (390px), pixel a pixel. As únicas diferenças restantes são o quadro do vídeo em reprodução, a foto da abertura no meio da troca e o endereço do rodapé, que agora mostra "Nº 15" como o cartão do mapa, porque os dois leem o mesmo endereço.